AWS Intern – Hands-On Task

Highly-Available Web Application

Step-by-Step Implementation Guide

Prepared by	Harshad
Organisation	CitiusCloud Services LLP
AWS Region	ap-south-1 (Mumbai)
Date Completed	May 16, 2026
GitHub Repository	github.com/harshad8782/AWS-Highly-Available-Web-Application-

Overview

This document provides a complete step-by-step guide to building a production-style, highly available web application on AWS. The infrastructure spans two Availability Zones and demonstrates VPC networking, EC2 compute, Application Load Balancing, Auto Scaling, and security best practices.

Objectives

- Design and provision a multi-AZ VPC with public/private subnet separation
- Deploy an Application Load Balancer (ALB) across two Availability Zones
- Configure an Auto Scaling Group (ASG) for self-healing and high availability
- Apply least-privilege security group rules (EC2 accepts traffic from ALB only)
- Validate all 5 evaluation checks and perform complete resource teardown

Architecture Summary

VPC	intern-vpc | 10.0.0.0/16 | vpc-0b6f0849eb8944559
Public Subnets	public-subnet-1 (10.0.1.0/24, ap-south-1a) | public-subnet-2 (10.0.2.0/24, ap-south-1b)
Private Subnets	private-subnet-1 (10.0.3.0/24, ap-south-1a) | private-subnet-2 (10.0.4.0/24, ap-south-1b)
Internet Gateway	intern-igw | igw-08fb9c85d671a314d
NAT Gateway	intern-nat | nat-0d637a35415e504d1 | EIP: 13.203.166.194
Load Balancer	intern-alb | Internet-facing | Active
ALB DNS	intern-alb-184512793.ap-south-1.elb.amazonaws.com
Target Group	intern-tg | HTTP:80 | Health check: GET/ every 30s
Auto Scaling Group	intern-asg | Min: 2 | Desired: 2 | Max: 4
EC2 Instances	t3.micro | Amazon Linux 2023 | NGINX | Private subnets
Security Groups	alb-sg (HTTP:80 from internet) | ec2-sg (HTTP:80 from alb-sg only)

Part 1 – VPC & Networking

The VPC forms the foundational network layer. All resources are isolated inside this custom VPC and public/private subnet separation enforces security boundaries.

Step 1: Create Custom VPC

Navigate to: AWS Console → VPC → Your VPCs → Create VPC

VPC Name	intern-vpc
IPv4 CIDR Block	10.0.0.0/16
Tenancy	Default
DNS Resolution	Enabled
VPC ID (created)	vpc-0b6f0849eb8944559
State	Available

Step 2: Create 4 Subnets

Navigate to: VPC → Subnets → Create subnet. Create all 4 subnets selecting intern-vpc as the VPC.

Subnet Name	Type	Availability Zone	CIDR Block
public-subnet-1	Public	ap-south-1a	10.0.1.0/24
public-subnet-2	Public	ap-south-1b	10.0.2.0/24
private-subnet-1	Private	ap-south-1a	10.0.3.0/24
private-subnet-2	Private	ap-south-1b	10.0.4.0/24

Important: Enable Auto-Assign Public IP on Public Subnets Only

Select public-subnet-1 → Actions → Edit subnet settings

Check: Enable auto-assign public IPv4 address → Save

Repeat the same for public-subnet-2

Do NOT enable this for private subnets — this is what makes them private

Step 3: Create & Attach Internet Gateway

Navigate to: VPC → Internet Gateways → Create internet gateway

1. Name: intern-igw → Click Create internet gateway
2. Select intern-igw → Actions → Attach to VPC
3. Select intern-vpc → Click Attach internet gateway

IGW ID	igw-08fb9c85d671a314d
Name	intern-igw

State	Attached
VPC	intern-vpc (vpc-0b6f0849eb8944559)

Step 4: Create NAT Gateway

The NAT Gateway allows private EC2 instances to download packages (NGINX) from the internet without being publicly accessible. It must be placed in a public subnet.

Navigate to: VPC → NAT Gateways → Create NAT Gateway

4. Name: intern-nat
5. Subnet: public-subnet-1 (CRITICAL — must be a public subnet)
6. Connectivity type: Public
7. Click Allocate Elastic IP → this assigns a public IP to the NAT
8. Click Create NAT Gateway → wait ~2 minutes for status: Available

NAT Gateway ID	nat-0d637a35415e504d1
Name	intern-nat
Subnet	public-subnet-1 (subnet-01c5b311790090f82)
Connectivity Type	Public
Public IP (EIP)	13.203.166.194
Private IP	10.0.1.56
State	Available

Step 5: Configure Route Tables

Route tables tell each subnet where to send traffic. Public subnets route to the IGW (internet-facing), private subnets route to the NAT Gateway (outbound only).

5a – Public Route Table

9. VPC → Route Tables → Create route table: Name: public-rt | VPC: intern-vpc
10. Select public-rt → Routes tab → Edit routes → Add route
11. Destination: 0.0.0.0/0 | Target: Internet Gateway → intern-igw → Save
12. Subnet associations tab → Edit subnet associations
13. Select public-subnet-1 and public-subnet-2 → Save associations

5b – Private Route Table

14. Create route table: Name: private-rt | VPC: intern-vpc
15. Routes → Edit routes → Add route
16. Destination: 0.0.0.0/0 | Target: NAT Gateway → intern-nat → Save
17. Subnet associations → Select private-subnet-1 and private-subnet-2 → Save

Route Table	Route Target	Associated Subnets

public-rt	0.0.0.0/0 → Internet Gateway	public-subnet-1, public-subnet-2
private-rt	0.0.0.0/0 → NAT Gateway	private-subnet-1, private-subnet-2

Part 2 – Security Groups

Security groups act as virtual firewalls. Two groups are created following the least-privilege model: one for the ALB (accepts internet traffic) and one for EC2 (accepts traffic from ALB only).

Step 6: Create ALB Security Group (alb-sg)

Navigate to: EC2 → Security Groups → Create security group

Name	alb-sg
Description	Allow HTTP from internet
VPC	intern-vpc
Security Group ID	sg-0c0779674fdacb646

Direction	Type	Protocol	Port	Source
Inbound	HTTP	TCP	80	0.0.0.0/0 (Anywhere)
Outbound	All traffic	All	All	0.0.0.0/0

Step 7: Create EC2 Security Group (ec2-sg)

This is the most critical security configuration. The EC2 security group references alb-sg as its source, meaning EC2 instances only accept HTTP traffic that comes through the ALB — not from the open internet.

Name	ec2-sg
Description	Allow HTTP from ALB only
VPC	intern-vpc

Direction	Type	Protocol	Port	Source
Inbound	HTTP	TCP	80	sg-0c0779674fdacb646 (alb-sg)
Outbound	All traffic	All	All	0.0.0.0/0

Why No SSH Rule?
SSH (port 22) is intentionally NOT added to ec2-sg
This proves private subnets are truly unreachable from the internet
Even if someone tries ssh ec2-user@<private-ip> from their laptop, it times out This satisfies Evaluation Check 3 and Check 4

Part 3 – EC2 Launch Template

The Launch Template defines the configuration for all EC2 instances launched by the Auto Scaling Group. It includes the AMI, instance type, security group, and NGINX installation script.

Step 8: Create Launch Template

Navigate to: EC2 → Launch Templates → Create launch template

Template Name	intern-lt
AMI	Amazon Linux 2023 (al2023-ami-*)
Instance Type	t3.micro (free tier eligible)
Key Pair	None — private subnet instances, no direct SSH needed
Security Group	ec2-sg
Subnet	Not set here — ASG handles placement

User Data Script

Paste this in EC2 → Launch Template → Advanced details → User data. It installs NGINX and creates a custom webpage showing the Instance ID and AZ.

```
#!/bin/bash
yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx

# IMDSv2 token for secure metadata access
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# Fetch instance metadata
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id)
AZ=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/placement/availability-zone)

# Write custom webpage
cat > /usr/share/nginx/html/index.html << 'HTMLEOF'
<html><body style='background:#1a1a2e;color:white;text-align:center;padding:60px'>
  <h1>CitiusCloud HA Web App</h1>
  <p>Instance: <b>INSTANCE_ID_HERE</b></p>
  <p>Zone: <b>AZ_HERE</b></p>
  <p>Refresh to see load balancing!</p>
</body></html>
HTMLEOF

# Replace placeholders with actual metadata
sed -i "s/INSTANCE_ID_HERE/$INSTANCE_ID/g" /usr/share/nginx/html/index.html
sed -i "s/AZ_HERE/$AZ/g" /usr/share/nginx/html/index.html
```

Part 4 – Application Load Balancer

The ALB distributes incoming HTTP traffic across EC2 instances in both Availability Zones. It is deployed in the public subnets and forwards traffic to the intern-tg target group.

Step 9: Create Target Group

Navigate to: EC2 → Target Groups → Create target group

Target Group Name	intern-tg
Target Type	Instances
Protocol	HTTP
Port	80
VPC	intern-vpc
Health Check Protocol	HTTP
Health Check Path	/
Health Check Interval	30 seconds
Healthy Threshold	2 consecutive successes
Unhealthy Threshold	2 consecutive failures
ARN	arn:aws:elasticloadbalancing:ap-south-1:361809382195:targetgroup/intern-tg/a5b8432a0fd8d954

Note: Do Not Register Targets Manually
Leave the registered targets empty at this stage
The Auto Scaling Group will automatically register EC2 instances into this target group
Manually registered instances would not be managed by ASG

Step 10: Create Application Load Balancer

Navigate to: EC2 → Load Balancers → Create Load Balancer → Application Load Balancer

Name	intern-alb
Load Balancer Type	Application
Scheme	Internet-facing
IP Address Type	IPv4
AZ 1	ap-south-1a → public-subnet-1
AZ 2	ap-south-1b → public-subnet-2
Security Group	alb-sg (remove default)
Listener	HTTP:80 → Forward to intern-tg
DNS Name	intern-alb-184512793.ap-south-1.elb.amazonaws.com

Status	Active

Part 5 – Auto Scaling Group

The Auto Scaling Group ensures a minimum of 2 EC2 instances are always running across private subnets. It monitors instance health via the ALB target group and automatically replaces any failed instance.

Step 11: Create Auto Scaling Group

Navigate to: EC2 → Auto Scaling Groups → Create Auto Scaling group

Configuration

18. Name: intern-asg | Launch template: intern-lt → Next
19. VPC: intern-vpc | Subnets: select private-subnet-1 AND private-subnet-2 → Next
20. Load balancing: Attach to existing load balancer → intern-tg
21. Enable ELB health checks → Next
22. Desired: 2 | Minimum: 2 | Maximum: 4 → Next → Create

ASG Name	intern-asg
Launch Template	intern-lt
Subnets	private-subnet-1, private-subnet-2
Min Size	2
Max Size	4
Desired Capacity	2
Target Group	intern-tg
Health Check Type	ELB
Health Check Grace Period	120 seconds

After Creation — Verify Instances Are Launched
Go to EC2 → Instances — you should see 2 new instances in Running state
One in ap-south-1a and one in ap-south-1b
Go to Target Groups → intern-tg → Targets — both should show Healthy
This may take 2-3 minutes while NGINX installs via user data

Part 6 – Validation & Testing

All 5 evaluation checks were completed and verified. Evidence is documented with screenshots in the GitHub repository.

#	Check	How to Verify	Result
1	Load balancing works	Open ALB DNS in browser, refresh multiple times — different Instance IDs and AZs appear	**PASS**
2	ASG self-healing	Manually terminated one EC2 → ASG launched replacement within 3 minutes automatically	**PASS**
3	Private subnets are private	SSH from laptop to private IP → Connection timed out (no public IP, no SSH rule)	**PASS**
4	Least-privilege networking	ec2-sg inbound shows port 80 from alb-sg (SG reference), not 0.0.0.0/0	**PASS**
5	Complete teardown	All resources deleted in correct order, no running instances or active NAT Gateway	**PASS**

Check 1: Load Balancing

23. Copy DNS: intern-alb-184512793.ap-south-1.elb.amazonaws.com
24. Open in browser → refresh 4-5 times
25. Different Instance IDs and AZs appear each refresh — confirms round-robin load balancing

Check 2: ASG Self-Healing

26. EC2 → Instances → select one instance → Instance State → Terminate
27. Wait 3-4 minutes
28. EC2 → Instances → a brand new instance appears automatically
29. ASG maintained desired capacity of 2 without manual intervention

Check 3: Private Subnets Are Private

```
# Run from your laptop terminal
$ ssh ec2-user@10.0.3.x

# Expected result (after ~30 seconds):
ssh: connect to host 10.0.3.x port 22: Connection timed out

# This proves:
# 1. Instance has no public IP address
# 2. No SSH inbound rule in ec2-sg
# 3. Private subnet has no IGW route
```

Check 4: Least-Privilege Security Groups

30. EC2 → Security Groups → ec2-sg → Inbound rules tab
31. Inbound shows: Type: HTTP | Port: 80 | Source: sg-0c0779674fdacb646 (alb-sg)
32. Source is the ALB security group ID — not 0.0.0.0/0 — confirming least privilege

Part 7 – CloudFormation Template

The entire infrastructure can be deployed automatically using a CloudFormation template, demonstrating Infrastructure as Code (IaC) skills. The template is available in the GitHub repository at CLOUDFORMATION TEMPLATE/intern-ha-webapp.yaml.

How to Deploy

33. AWS Console → CloudFormation → Create stack → Upload a template file
34. Upload: intern-ha-webapp.yaml
35. Stack name: intern-ha-stack → Next → Next → Create stack
36. Wait ~5 minutes for all resources to provision
37. Find the ALB DNS name in the Outputs tab → open in browser

Resources Created by Template

- VPC (10.0.0.0/16) with DNS enabled
- 4 Subnets — 2 public, 2 private across ap-south-1a and ap-south-1b
- Internet Gateway attached to VPC
- NAT Gateway with Elastic IP in public-subnet-1
- Public and private route tables with correct associations
- ALB Security Group and EC2 Security Group (least privilege)
- EC2 Launch Template with NGINX user data
- Application Load Balancer, Target Group, and Listener
- Auto Scaling Group with ELB health checks

Part 8 – Teardown Checklist

Resources must be deleted in the exact order below to avoid dependency errors. Skipping steps or deleting out of order will result in errors.

Warning: Delete in This Exact Order

Deleting out of order causes dependency errors (e.g. cannot delete VPC if subnets exist)

NAT Gateway takes ~5 minutes to fully delete — wait before releasing Elastic IP

Always verify no running instances or allocated EIPs remain after teardown

#	Resource	Steps
1	Auto Scaling Group (intern-asg)	EC2 → Auto Scaling Groups → Delete → Force delete. Wait for instances to terminate
2	Application Load Balancer (intern-alb)	EC2 → Load Balancers → Actions → Delete
3	Target Group (intern-tg)	EC2 → Target Groups → Actions → Delete
4	Launch Template (intern-lt)	EC2 → Launch Templates → Actions → Delete
5	NAT Gateway (intern-nat)	VPC → NAT Gateways → Actions → Delete. WAIT 5 MINUTES
6	Elastic IP	VPC → Elastic IPs → Actions → Release address
7	Internet Gateway (intern-igw)	Detach from VPC first, then Delete
8	All 4 Subnets	VPC → Subnets → Delete each subnet
9	Route Tables	Delete public-rt and private-rt (not the main/default)
10	Security Groups	Delete ec2-sg first, then alb-sg
11	VPC (intern-vpc)	VPC → Your VPCs → Actions → Delete VPC

Verify Complete Teardown

- EC2 → Instances: no running instances
- VPC → NAT Gateways: none in Available state
- VPC → Elastic IPs: none allocated
- EC2 → Load Balancers: none listed
- VPC → Your VPCs: intern-vpc no longer present

Summary

This assignment successfully demonstrated end-to-end AWS infrastructure skills by building a fully functional, production-style highly available web application. All 5 evaluation checks passed.

Skill Demonstrated	What Was Built
Multi-AZ VPC Networking	VPC + 4 subnets + IGW + NAT + Route Tables
Least-Privilege Security	alb-sg + ec2-sg referencing each other
Load Balancing	ALB across ap-south-1a and ap-south-1b
Auto Scaling & Self-Healing	ASG min:2 with ELB health checks
Infrastructure as Code	CloudFormation template for full automation
Cost Discipline	Complete teardown in correct order

GitHub Repository: github.com/harshad8782/AWS-Highly-Available-Web-Application-

Harshad | ap-south-1 (Mumbai) | May 2026